Q. What is software project management? Explain its importance in IT industry. [8 marks]
→
Software Project Management (SPM) is the discipline of **planning, organizing, and managing software projects** to ensure that software products are delivered **on time, within budget, and with required quality**.
SPM focuses on coordinating all aspects of a software project such as:

- Resource management
- Risk management
- Cost estimation
- Change control

It ensures proper alignment between **technical development** and **business objectives**, making the project successful.

Importance of SPM in IT Industry

1. Handles Complexity of Software Development

- Software projects are **complex, evolving, and unpredictable**
- SPM provides **structure and systematic approach** to manage this complexity

2. Ensures Timely Delivery

- Helps in proper **planning and scheduling of tasks**
- Prevents delays and ensures project completion **within deadlines**

3. Controls Cost and Budget

- Includes **cost estimation and budgeting techniques**
- Prevents **cost overruns**, which is a common issue in IT projects

4. Improves Quality of Software

- Incorporates **testing, quality assurance, and monitoring**
- Ensures the final product meets **user requirements and standards**

5. Effective Risk Management

- Identifies potential risks early
- Applies **mitigation strategies** to reduce project failure chances

6. Enhances Communication and Coordination

- Improves communication among:
  - Developers
  - Stakeholders
  - Project managers
- Reduces misunderstandings and conflicts

7. Ensures Stakeholder Satisfaction

- Keeps stakeholders involved throughout the project
- Ensures the final product meets **user expectations**

8. Increases Project Success Rate

- As per notes, many projects fail due to poor management
- SPM increases chances of success by applying **proper planning and control**

github.com/marsh4dev

Q. Differentiate between software project management and general project management. [7 marks]
→
Software Project Management (SPM) deals with managing **software development projects**, while General Project Management deals with **traditional projects** like construction or manufacturing.

Software Project Management vs General Project Management

Aspect	General Project Management	Software Project Management
Nature of Product	Tangible, physical product (e.g., building, machine)	Intangible product (software, code, functionality)
Requirements Stability	Mostly stable after planning	Frequently changing and evolving
Flexibility to Change	Low flexibility; changes are costly and difficult	High flexibility; changes can be made even late
Resource Focus	Uses both human resources and physical materials	Mainly human resources (developers, testers)
Approach / Lifecycle	Linear, sequential (e.g., Waterfall)	Iterative and incremental (e.g., Agile)
Quality Assurance	Testing done at specific milestones	Continuous testing and improvement
Progress Visibility	Visible progress (e.g., building structure)	Less visible, tracked via features and milestones
Stakeholder Involvement	Mostly at beginning and end	Continuous involvement throughout development

github.com/harsh4dev

Q. What is a Work Breakdown Structure (WBS)? Why is it essential in software project management? [7 marks]

→

Work Breakdown Structure (WBS) is a **hierarchical decomposition of a project into smaller, manageable tasks or components**, which helps in planning and execution of the project effectively.

Why WBS is Essential in Software Project Management

1. Better Project Planning

- Divides complex project into **smaller manageable parts**
- Makes planning more **accurate and systematic**

2. Clear Task Allocation

- Helps in assigning tasks to **team members clearly**
- Avoids confusion and overlapping responsibilities

3. Improved Time Estimation

- Each small task can be **estimated more precisely**
- Leads to better **schedule management**

4. Cost Estimation and Control

- Enables accurate **cost estimation for each task**
- Helps in controlling overall project budget

5. Easy Monitoring and Control

- Progress can be tracked at **task level**
- Makes it easier to identify **delays and issues**

6. Reduces Project Complexity

- Simplifies large projects by breaking them into **smaller units**
- Makes development more **manageable**

7. Improves Communication

- Provides a clear structure of work
- Enhances understanding among **team members and stakeholders**

github.com/harsh4dev

Q. What is project scope? How is it defined and managed in software projects? [8 marks]
→
Project scope refers to the **detailed description of all the work, deliverables, features, and functionalities** that are included in a software project.
It clearly defines **what is to be done and what is not to be done** in the project.
Explanation:

- It sets the **boundaries of the project**
- Helps stakeholders understand the **objectives and expected outcomes**
- Prevents unnecessary work outside the defined requirements (**scope creep**)

How Project Scope is Defined in Software Projects
1. Requirement Gathering

- Collect requirements from stakeholders and users
- Understand system needs, features, and expectations

2. Scope Statement Preparation

- Document the project scope clearly
- Includes:
    - Project objectives
    - Deliverables
    - Features and functionalities
    - Constraints and assumptions

3. Work Breakdown Structure (WBS)

- Break the scope into **smaller manageable tasks**
- Helps in better planning and execution

4. Scope Validation

- Review and approve scope with stakeholders
- Ensures agreement on what will be delivered

How Project Scope is Managed
1. Scope Planning

- Define how scope will be controlled and monitored
- Establish guidelines for handling changes

2. Scope Control

- Monitor project activities to ensure they align with scope
- Prevent **scope creep** (uncontrolled changes)

3. Change Management

- Any change in scope must follow a **formal approval process**
- Evaluate impact on **time, cost, and resources**

4. Continuous Monitoring

- Regularly track progress against defined scope
- Ensure project stays within defined boundaries

github.com/harsh4dev

Q. Explain the concept of milestones with examples of major and minor milestones. [8 marks]
→

Milestones are **significant checkpoints or key events in a project timeline** that indicate the completion of an important phase or activity.

- Milestones do **not represent work**, but they mark the **completion of work**
- They are used to **track progress and evaluate performance**
- Help project managers ensure that the project is **on schedule**

Major Milestones

These represent the **completion of major phases** of the project.

Examples:

- Completion of **requirement analysis**
- Approval of **system design**
- Completion of **coding phase**
- Final **system testing completed**
- **Project delivery / deployment**

These milestones are **critical** and usually reviewed by stakeholders.

Minor Milestones

These represent the **completion of smaller tasks or sub-activities** within a phase.

Examples:

- Completion of a **specific module (e.g., login system)**
- Database design finalized
- UI design of a single page completed
- Unit testing of a module finished

These milestones help in **detailed tracking of progress**

Importance of Milestones

1. Progress Tracking

- Helps in monitoring whether the project is **on track**

2. Better Planning and Scheduling

- Divides project timeline into **manageable checkpoints**

3. Performance Evaluation

- Helps evaluate team performance at different stages

4. Stakeholder Communication

- Provides clear updates to stakeholders about **project status**

github.com/harsh4dev

Q. What is Agile methodology in software project management? How does it differ from traditional approaches? [8 marks]

→

Agile methodology is an **iterative and incremental approach** to software development where requirements and solutions evolve through **continuous collaboration, feedback, and flexibility**.

 In Agile, the project is divided into **small iterations (sprints)**
 Each iteration delivers a **working part of the software**
 Emphasizes:

- Customer collaboration
- Continuous improvement
- Quick response to changes

Difference between Agile and Traditional Approaches

Aspect	**Traditional Approach (e.g., Waterfall)**	**Agile Methodology**
Process Type	Linear and sequential	Iterative and incremental
Flexibility	Low (changes are difficult)	High (changes are easily accepted)
Requirement Handling	Fixed at the beginning	Evolving throughout the project
Customer Involvement	Limited (start and end)	Continuous involvement
Delivery	Single final delivery	Frequent small deliveries
Testing	Done at the end	Continuous testing in each iteration
Risk Management	Risks identified late	Risks identified early
Focus	Process-oriented	Customer satisfaction and working software

Agile methodology is more **flexible, adaptive, and customer-focused** compared to traditional approaches, making it highly suitable for modern software development where requirements frequently change.

github.com/harsh4dev

Q. Explain the Scrum framework and its key roles. [7 marks]
→

Scrum is an **Agile framework** used for managing and developing software projects through **iterative and incremental processes called sprints**.

Explanation:

- Work is divided into **small time-boxed iterations (Sprints)** (usually 2–4 weeks)
- Each sprint delivers a **working increment of the product**
- Emphasizes:
  - Team collaboration
  - Continuous feedback
  - Rapid delivery

Key Roles in Scrum

1. Product Owner

- Represents the **customer/stakeholders**
- Responsible for managing the **Product Backlog**
- Defines and prioritizes requirements
- Ensures the team delivers **maximum business value**

2. Scrum Master

- Acts as a **facilitator and guide**
- Ensures Scrum practices are followed properly
- Removes obstacles faced by the team
- Helps improve team productivity

3. Development Team

- Group of professionals (developers, testers, designers)
- Responsible for **designing, coding, and testing**
- Self-organizing and cross-functional
- Delivers a **working product increment** at the end of each sprint

Scrum is an effective Agile framework that improves **team collaboration, flexibility, and continuous delivery**, with clearly defined roles ensuring smooth project execution.

github.com/harsh4dev

Q. What is risk management in software projects? Explain the steps involved in the risk management process. [7 marks]

→

Risk management is the process of **identifying, analyzing, and controlling potential risks** that may negatively affect a software project.

Explanation:

- A risk is any **uncertain event** that can impact:
  - Time
  - Cost
  - Quality
- Risk management helps in **minimizing the impact of problems** and improving project success.

Steps in Risk Management Process

1. Risk Identification

- Identify possible risks that may affect the project
- Examples:
  - Requirement changes
  - Technical issues
  - Resource shortage

2. Risk Analysis

- Evaluate the **probability and impact** of each risk
- Helps prioritize risks based on severity

3. Risk Prioritization

- Rank risks as **high, medium, or low**
- Focus more on **critical risks**

4. Risk Planning (Mitigation Planning)

- Develop strategies to **reduce or eliminate risks**
- Example:
  - Backup resources
  - Use proven technologies

5. Risk Monitoring and Control

- Continuously track identified risks
- Take corrective actions if risks occur

Risk management is essential in software projects as it helps in **anticipating problems, reducing uncertainties, and ensuring smooth project execution**, leading to successful project completion.

github.com/harsh4dev

Q. What is Earned Value Management (EVM)? Explain how it is used to monitor project performance. [8 marks]

→

Earned Value Management (EVM) is a **project performance measurement technique** that integrates **scope, time, and cost** to assess the progress and performance of a software project.

Explanation:

- EVM compares:
  - **Planned work**
  - **Actual work completed**
  - **Actual cost incurred**
- It helps project managers determine whether the project is **on schedule and within budget**

Key Components of EVM

1. Planned Value (PV)

- The **budgeted cost** of work scheduled to be completed

2. Earned Value (EV)

- The **budgeted cost** of actual work completed

3. Actual Cost (AC)

- The **actual cost incurred** for the work completed

How EVM is Used to Monitor Project Performance

1. Schedule Performance

- Compare EV with PV
- If **EV < PV → Project is behind schedule**
- If **EV > PV → Project is ahead of schedule**

2. Cost Performance

- Compare EV with AC
- If **EV < AC → Over budget**
- If **EV > AC → Under budget**

3. Performance Indicators

a) Schedule Variance (SV)

- SV = EV − PV
- Indicates schedule status

b) Cost Variance (CV)

- CV = EV − AC
- Indicates cost performance

c) Schedule Performance Index (SPI)

- SPI = EV / PV
- SPI < 1 → Behind schedule
- SPI > 1 → Ahead of schedule

d) Cost Performance Index (CPI)

- CPI = EV / AC
- CPI < 1 → Over budget
- CPI > 1 → Under budget

github.com/harsh4dev

Q. What are software metrics? Explain their importance in measuring project success. [7 marks]
→

Software metrics are **quantitative measures** used to evaluate different aspects of a software project such as **size, quality, performance, cost, and productivity**.
They help in **monitoring, controlling, and improving** the software development process.

Importance of Software Metrics in Measuring Project Success

Software metrics play a crucial role in ensuring that a project is completed **on time, within budget, and with desired quality** (as emphasized in SPM goals)

1. Performance Measurement

- Metrics help measure **progress of the project**.
- Example: Tracking completed tasks vs planned tasks.

Helps determine whether the project is **on schedule or delayed**.

2. Cost and Effort Control

- Metrics provide data on **effort estimation and cost usage**.
- Helps answer:
    - How much cost is spent?
    - Are we exceeding the budget?

Supports **cost management and budgeting control**.

3. Quality Assurance

- Metrics such as **defect density, error rate** help measure software quality.
- Ensures that the product meets **required standards**.

Leads to **better reliability and user satisfaction**.

4. Better Decision Making

- Managers use metrics for **planning and adjustments**.
- Helps in:
    - Resource allocation
    - Risk handling

Enables **data-driven decisions instead of assumptions**.

5. Risk Identification and Control

- Metrics highlight **deviations and potential risks early**.
- Example: Increasing defect rate indicates quality issues.

Helps in **early risk mitigation**, a key SPM objective.

6. Productivity Evaluation

- Measures team performance (e.g., output per developer).
- Helps identify **efficient and inefficient processes**.

Improves **team productivity and efficiency**.

7. Project Monitoring and Control

- Provides continuous tracking of:
    - Time
    - Cost
    - Scope

Ensures **project stays aligned with goals and stakeholder expectations**.

github.com/harsh4dev

Q. Describe the ethical and legal considerations in software project management. [8 marks]
→
Ethical and legal considerations in Software Project Management ensure that software is developed and delivered in a **responsible, fair, and lawful manner**. They protect **users, stakeholders, and organizations** while ensuring project success.

A. Ethical Considerations
Ethics refers to the **moral principles** guiding the behavior of project teams.
1. Honesty and Transparency
- Project managers must provide **accurate information** about cost, schedule, and risks.

2. Data Privacy and Confidentiality
- Protect sensitive user data (personal, financial, health).

3. Quality Responsibility
- Deliver software that is **reliable and safe**.

4. Respect for Intellectual Property
- Do not copy code, designs, or content without permission.

5. Fair Treatment of Team Members
- Ensure equal opportunities and avoid discrimination.

6. Social Responsibility
- Software should not harm society (e.g., misuse of data, unethical AI).

B. Legal Considerations
Legal aspects ensure compliance with **laws and regulations**.
1. Intellectual Property Rights (IPR)
- Includes copyrights, patents, and trademarks.
- Software must not violate ownership rights.

2. Data Protection Laws
- Follow laws related to data privacy (e.g., user data protection policies).

3. Contract Management
- Projects must follow legal agreements between Clients, Vendors, and Stakeholders

4. Licensing Compliance
- Use software tools and libraries according to their licenses (open-source or commercial).

5. Liability and Accountability
- Define who is responsible if the software fails or causes damage.

6. Regulatory Compliance
- Follow industry standards and government regulations.
- Example: Financial systems must follow central bank guidelines.

Ethical and legal considerations are essential in Software Project Management to ensure **trust, fairness, and compliance**.
Ignoring them can lead to **project failure, legal penalties, and loss of reputation**, while proper adherence ensures **successful and responsible software delivery**.

github.com/harsh4dev

Q. Describe how cultural shifts are managed when transitioning to modern software management processes. [8 marks]

→

Cultural shift refers to the **change in mindset, values, and working practices** when an organization moves from traditional methods (e.g., Waterfall) to **modern approaches** (e.g., Agile, DevOps).

Managing this shift is essential for **successful adoption and project performance**.

Key Ways to Manage Cultural Shifts

1. Strong Leadership and Vision

- Management must clearly communicate the **need for change** and its benefits.

2. Training and Skill Development

- Provide training on modern methodologies like Agile, Scrum, DevOps tools

3. Effective Communication

- Maintain open and transparent communication.
- Explain Why change is needed and how it affects team members.

4. Gradual Implementation (Incremental Change)

- Introduce changes step-by-step instead of all at once.
- Start with pilot projects.

Minimizes **risk and resistance to change**.

5. Encouraging Collaboration and Teamwork

- Promote cross-functional teams.
- Break down hierarchical barriers.

Supports **Agile culture of collaboration and flexibility**.

6. Change Management Strategies

- Use structured approaches like: Planning change, Monitoring progress, Handling resistance

7. Reward and Recognition

- Appreciate employees who adopt new practices.
- Provide incentives for learning and improvement.

8. Continuous Feedback and Improvement

- Collect feedback from team members regularly.
- Adapt processes based on feedback.

Managing cultural shifts requires **leadership, communication, training, and gradual adaptation**.

A well-managed transition ensures **better collaboration, higher productivity, and successful implementation of modern software management practices**.

github.com/harsh4dev

Q. Explain the process of project closure and its significance. [8 marks]
→
Project closure is the **final phase of the project lifecycle** where all project activities are completed, deliverables are handed over, and the project is formally **closed and evaluated**.

Process of Project Closure

1. Completion of Deliverables
- Ensure all project requirements and objectives are fully achieved and met agreements.

2. Client Acceptance and Handover
- Deliver the completed software to the client.
- Obtain **formal acceptance/sign-off** from stakeholders.

3. Documentation and Reporting
- Prepare and finalize all documents such as Project reports User manuals, and technical documentation.

4. Financial Closure
- Settle all project-related payments and expenses.
- Close contracts with vendors and suppliers.

5. Resource Release
- Release project team members and resources.
- Reassign them to other projects.

6. Performance Evaluation
- Compare actual results with planned Time, Cost, and Quality

7. Lessons Learned
- Identify What went well, and What problems occurred
- Document best practices and mistakes.

Significance of Project Closure

1. Ensures Formal Completion
- Confirms that all tasks are completed and no pending work remains.

2. Improves Future Projects
- Lessons learned help avoid mistakes and improve planning.

3. Accountability and Transparency
- Proper documentation ensures clarity and responsibility.

4. Client Satisfaction
- Formal acceptance ensures that client expectations are met.

5. Efficient Resource Utilization
- Frees resources for new projects.

6. Financial Control
- Ensures all financial matters are properly settled.

github.com/harsh4dev

Q. b. How are lessons learned documented and used in future projects? [7 marks]
→

Lessons learned are the **knowledge gained from project experiences**, including both **successes and failures**, which are documented and used to improve future projects.

A. How Lessons Learned are Documented

1. Conducting Review Meetings

- At the end of the project (or phases), teams hold meetings and discuss what went well and what did not.

2. Identifying Key Points

- Document: Success factors, Problems faced, Risks encountered, Solutions applied

3. Recording in a Structured Format

- Lessons are written in documents such as:
  - Lessons Learned Report
  - Project Closure Report

4. Categorization

- Organize lessons into categories like:
  - Technical issues
  - Management issues
  - Resource-related problems

5. Storing in Knowledge Repository

- Store documents in a centralized system/database.

B. How Lessons Learned are Used in Future Projects

1. Improving Project Planning

- Helps in better estimation of:
  - Time
  - Cost
  - Resources

2. Risk Management

- Previously identified risks help in:
  - Early detection
  - Better mitigation strategies

3. Process Improvement

- Enhances development practices and workflows.

4. Better Decision Making

- Managers use past experiences to make informed decisions.

5. Training and Knowledge Sharing

- Used as learning material for new team members.

6. Avoiding Repetition of Mistakes

- Prevents recurring issues in future projects.

github.com/harsh4dev

Q. What is Software Project Management? Explain the key features of conventional software management. (8 marks)

→

Software Project Management is the process of **planning, organizing, directing, and controlling** software development activities to achieve specific objectives within **time, cost, and quality constraints**.

It ensures that the software project is completed **efficiently and successfully**.

Key Features of Conventional Software Management

Conventional (traditional) software management follows a **rigid, plan-driven approach** (commonly associated with Waterfall model).

1. Sequential Development Approach

- Follows a linear sequence:
  - Requirements → Design → Implementation → Testing → Maintenance

2. Heavy Documentation

- Emphasis on detailed documentation at every stage.
- Includes:
  - Requirement documents
  - Design specifications

3. Fixed Requirements

- Requirements are defined at the beginning and rarely changed later.

4. Strict Planning and Control

- Detailed project plan is created in advance.
- Progress is strictly monitored against the plan.

5. Limited Customer Involvement

- Customer involvement is mostly during:
  - Initial requirement phase
  - Final delivery

6. Late Testing Phase

- Testing is performed after development is completed.

7. Risk of Failure in Dynamic Environments

- Not suitable for projects with changing requirements.

8. Clearly Defined Roles and Responsibilities

- Each team member has a specific role.
- Hierarchical structure is followed.

Software Project Management ensures successful delivery of software within constraints, while conventional management provides a **structured and controlled approach**, but lacks **flexibility and adaptability** compared to modern methods.

github.com/harsh4dev

Q. What are the key principles of modern software management? Explain any four with suitable examples. [8 marks]

→

Modern software management focuses on **flexibility, adaptability, and continuous improvement**, unlike traditional rigid approaches. It is commonly associated with **Agile and iterative models**.

Key Principles

1. Iterative and Incremental Development

- Software is developed in **small increments (iterations)**.
- Each iteration delivers a working part of the system.

Example:

A mobile app is developed feature-by-feature (login → dashboard → payment), instead of building everything at once.

2. Continuous Feedback and Customer Involvement

- Customers are actively involved throughout development.
- Regular feedback is collected and incorporated.

Example:

After each sprint, users test the feature and suggest improvements, which are added in the next iteration.

3. Risk-Driven Approach

- High-risk elements are identified and addressed early.
- Focus on minimizing uncertainties.

Example:

If a project uses a new AI model, it is tested early to check feasibility before full development.

4. Emphasis on Quality (Continuous Testing)

- Testing is performed throughout the development process, not just at the end.

Example:

Developers run automated tests after every code update to detect bugs early.

5. Adaptability to Change

- Requirements can be modified even during development.

Example:

If client changes UI design mid-project, the team updates it in the next sprint.

6. Component-Based Development

- Software is built using reusable components or modules.

Example:

Using pre-built authentication modules instead of coding from scratch.

github.com/harsh4dev

Q. What is software economics? Discuss the five key that influence software cost estimation. (8 marks]

→

Software economics is the study of **cost, benefits, and value** involved in software development. It focuses on estimating, managing, and optimizing the **resources (time, effort, and money)** required to develop software.

Key Factors Influencing Software Cost Estimation

1. Project Size and Complexity

- Larger and more complex projects require more:
  - Time
  - Effort
  - Resources

2. Team Experience and Skill Level

- Skilled and experienced developers work more efficiently.
- Less experienced teams may take more time and introduce errors.

3. Technology and Tools Used

- Use of advanced tools, frameworks, or automation can:
  - Reduce development time
  - Increase initial cost (tools/licenses)

4. Requirements Stability

- Frequent changes in requirements increase:
  - Rework
  - Development time

5. Development Methodology

- Different models affect cost:
  - Waterfall → Predictable but rigid
  - Agile → Flexible but may involve continuous changes

6. Schedule Constraints

- Tight deadlines require:
  - More developers
  - Overtime work

7. Risk Factors

- Uncertain or high-risk projects require:
  - Extra planning
  - Backup solutions

github.com/harsh4dev

Q. What are the challenges in software cost estimation? Discuss the attributes of a good cost estimation. (8marks)

→

A. Challenges in Software Cost Estimation

1. Unclear/Changing Requirements

- Requirements are often incomplete at the beginning.
- Frequent changes lead to **rework and inaccurate estimates**.

2. Complexity of Software

- Software systems are intangible and complex.
- Difficult to measure size and effort precisely.

3. Lack of Historical Data

- Estimation becomes difficult without past project data.

4. Human Factors

- Developer productivity varies based on skill and experience.

5. Technological Changes

- New tools/technologies introduce uncertainty.

6. Time Pressure

- Estimations are often done quickly, reducing accuracy.

B. Attributes of a Good Cost Estimation

1. Accuracy

- Should be close to actual cost.

2. Realistic

- Must consider real-world constraints (time, resources).

3. Based on Data

- Uses historical data and proven models.

4. Flexible

- Can adapt to requirement changes.

5. Comprehensive

- Covers all cost factors (development, testing, maintenance).

6. Clear and Understandable

- Easy for stakeholders to interpret.

github.com/harsh4dev

Q. Configuration management in software projects (5 marks)
→
Configuration Management (CM) is the process of **identifying, organizing, and controlling changes** to software artifacts throughout the project lifecycle.

Key Activities:
1. Configuration Identification
- Identify items (code, documents, designs).

2. Version Control
- Maintain different versions of software.

3. Change Control
- Manage and approve changes systematically.

4. Configuration Status Accounting
- Track status of items and changes.

5. Configuration Audits
- Ensure consistency and correctness.

Q. Process groups and knowledge areas (5 marks)
→
A. Process Groups (PMBOK)
1. **Initiation** – Define project
2. **Planning** – Set goals and strategy
3. **Execution** – Perform project work
4. **Monitoring & Controlling** – Track progress
5. **Closing** – Complete project

B. Knowledge Areas
1. Integration Management
2. Scope Management
3. Time Management
4. Cost Management
5. Quality Management
6. Human Resource Management
7. Communication Management
8. Risk Management
9. Procurement Management

github.com/harsh4dev

Q. Role of a Project Manager in Agile vs. Traditional Environments. (5 marks)
→

Role of Project Manager in:

Agile Environment

- Acts as facilitator/servant leader
- Encourages team collaboration
- Manages sprints and backlog
- Adapts to changes quickly

Focus: Flexibility & team empowerment

Traditional Environment

- Acts as decision-maker and controller
- Responsible for:
    - Planning
    - Scheduling
    - Budgeting
- Strict monitoring of progress

Focus: Control & predictability

github.com/harsh4dev